

Portfolio Governance Bot

A dual-entry (Slack + HTML) chatbot answering natural-language portfolio-status questions — risk, slippage, governance level, open defects — against a live Google Sheet, via one LLM agent.

Problem

Portfolio managers re-read the same tracker repeatedly to answer the same handful of questions — what's at risk, what's slipped, what needs governance attention. This project builds a portfolio-governance pipeline: pulling project data from a live Google Sheet, reasoning over it with an AI agent, and answering on demand from either Slack or a standalone web form — so retrieval and synthesis stop being manual work.

Approach & Key Decisions

- n8n (low-code) over a custom backend — faster to iterate, at the cost of UI-driven bugs being harder to catch than compiler errors.
- Two fully parallel ingestion lanes (Slack, HTML) converging only at the shared Agent — avoids cross-branch reference errors when only one trigger fires per run.
- One shared Agent + system prompt serving both entry points — Slack and the HTML form normalize into the same shape so a single gpt-4o-mini Agent answers consistently either way.
- gpt-4o-mini, not a larger model — bounded structured Q&A doesn't need a frontier model; cost and latency mattered more.
- No authentication on either webhook — an explicit, acknowledged trade-off for a scoped demo, not an oversight.

What It Does

- Two independent triggers: Slack Events API webhook, and a standalone HTML form webhook.
- Handles Slack's URL-verification handshake and filters out bot/echo messages.
- Fetches a live 46-row governance sheet: 7 programs, Epic + Sub-task rows, baseline vs. current due dates, computed slippage, RAG status, rollout % complete, governance tier, known issues.
- One LangChain Agent (gpt-4o-mini) answers from that data only, in Slack markdown, under 400 words.
- Routes the answer back to its source: Slack reply in-thread, or JSON response to the browser (HTML-sourced questions also post to Slack).
- A workflow-level Error Trigger posts any node failure to a dedicated Slack alert channel.

Results & Learnings

- The dual-entry, single-agent design held up once wired correctly — one prompt, one model, two front doors, consistent answers.
- A webhook's Respond setting defaulting to “Immediately” silently overrides a correctly-built downstream response node — the single largest source of debugging time.
- A single HTTP request gets one response; a misleadingly-named duplicate response node won that race silently.
- Templating JSON as a raw string breaks on any multiline LLM output — build a real object expression and let the platform serialize it.
- A stray trailing '}}' in a copy-pasted IF condition silently broke source routing without ever throwing an error.

Tech Stack

Workflow engine	n8n Cloud
AI Agent / LLM	LangChain Agent node · OpenAI gpt-4o-mini
Data source	Google Sheets (n8n-nodes-base.googleSheets v4)
Chat integration	Slack Events API + Slack Web API (n8n-nodes-base.slack v2)
Frontend	Static HTML + vanilla JavaScript (fetch), no framework